Frontend Integration Guide: `MagizinePublic.Api` Reader Authentication & Session Lifecycle (Phase 1)

This guide covers the end-to-end integration of **Public Reader Authentication (`Tbl_Reader`)** in **`MagizinePublic.Api`** (`http://localhost:5080` / `https://localhost:7080`), including:

- **Selective Public vs. Protected Access**: Public article & category reading remains 100% open (`[AllowAnonymous]`), while Reader profile, Bookmark, and Author Subscription actions use authenticated Reader sessions.
- **Zero-Friction Registration (`POST /api/public/auth/Register`)**: Auto-activates the Reader (`IsActive = true`), auto-generates a unique `Username` if omitted, and immediately sets `HttpOnly` cookies (`reader_access_token` + `reader_refresh_token`) so the Reader is signed in right after registering.
- **Enterprise Concurrent Refresh (`POST /api/public/auth/Refresh`)**: Combines a **Frontend Mutex Queue Interceptor** with a **30-second Backend Grace Period (`PreviousRefreshTokenHash`)** for multi-tab safety.

---

1. Cookie & Token Architecture

Cookie / Token	Name	Scope (`Path`)	Lifetime	Security Flags
Access Token Cookie	`reader_access_token`	`/`	`60 minutes`	`HttpOnly`, `SameSite=Lax`, `Secure` (HTTPS)
Refresh Token Cookie	`reader_refresh_token`	`/api/public/auth`	`7 days`	`HttpOnly`, `SameSite=Strict`, `Secure` (HTTPS)
Backend Grace Window	`PreviousRefreshTokenHash`	Database (`dbo.Tbl_Reader`)	`30 seconds`	Prevents multi-tab refresh race conditions

> **Cookie Isolation on `localhost`:**
> - `MagizineAdmin.Api`: `admin_access_token` & `admin_refresh_token` (`Path=/api/admin/auth`)
> - `MagizineAuthor.Api`: `author_access_token` & `author_refresh_token` (`Path=/api/author/auth`)
> - `MagizinePublic.Api`: `reader_access_token` & `reader_refresh_token` (`Path=/api/public/auth`)

---

2. Default Seeded Reader Account (For Local Testing)

Field	Value
`ReaderId`	`01JREADER0000000000000001`
`Username`	`sophia.reader`
`Email`	`sophia.reader@ledition.com`
`Password`	`Admin@123`

---

3. Endpoint Reference (`/api/public/auth`)

3.1 `POST /api/public/auth/Register` (`[AllowAnonymous]`)

Registers a new Reader, sets `IsActive = true`, and immediately issues `reader_access_token` and `reader_refresh_token` `HttpOnly` cookies.

Request (`application/json`):

```
{
  "FirstName": "Claire",
  "LastName": "Moreau",
  "Username": "claire.moreau",
  "Email": "claire.moreau@ledition.com",
  "Password": "Reader@123",
  "ConfirmPassword": "Reader@123",
  "AvatarUrl": "https://images.unsplash.com/photo-1534528741775-53994a69daeb",
  "Bio": "Collector of contemporary design books and photography essays."
}
```

(Note: `Username` is optional—if omitted, the backend auto-generates a unique handle from `FirstName` + `LastName`, e.g. `claire.moreau`.)

Response (`200 OK`):

```
{
  "IsSuccess": true,
  "RespCode": "MS#000",
  "RespDesp": "Reader account registered and signed in successfully.",
  "RespType": "Success",
  "Data": {
    "AccessToken": "eyJhbGciOiJIUzI1NiIs...",
    "TokenType": "Bearer",
    "ExpiresAt": "2026-10-12T17:00:00+00:00",
    "RefreshTokenExpiresAt": "2026-10-19T16:00:00+00:00",
    "SessionId": "01J...",
    "IsGracePeriodHit": false,
    "Reader": {
      "ReaderId": "01J...",
      "FirstName": "Claire",
      "LastName": "Moreau",
      "FullName": "Claire Moreau",
      "Username": "claire.moreau",
      "Email": "claire.moreau@ledition.com",
      "AvatarUrl": "https://images.unsplash.com/photo-1534528741775-53994a69daeb",
      "Bio": "Collector of contemporary design books and photography essays.",
      "IsActive": true,
      "LastLoginAt": "2026-10-12T16:00:00+00:00",
      "CreatedAt": "2026-10-12T16:00:00+00:00",
      "UpdatedAt": null
    }
  }
}
```


3.2 `POST /api/public/auth/Login` (`[AllowAnonymous]`)

Authenticates by `Username` or `Email` + `Password` and sets `HttpOnly` cookies.


```
{
  "UsernameOrEmail": "sophia.reader",
  "Password": "Admin@123"
}
```


3.3 `POST /api/public/auth/Refresh` (`[AllowAnonymous]`)

Automatically reads the `reader_refresh_token` `HttpOnly` cookie (or optional `{ "RefreshToken": "..." }` JSON fallback), rotates the token, and preserves the previous token for `30 seconds` (`PreviousRefreshTokenHash`).

3.4 `GET /api/public/auth/Me` (`[Authorize]`)

Returns the current authenticated Reader's `ReaderProfileResModel`.

3.5 `POST /api/public/auth/UpdateProfile` (`[Authorize]`)

Updates `FirstName`, `LastName`, `Username`, `Email`, `AvatarUrl`, and `Bio`.

3.6 `POST /api/public/auth/ChangePassword` (`[Authorize]`)

Verifies `CurrentPassword`, sets `NewPassword`, revokes all active refresh tokens in `dbo.Tbl_Reader`, and clears auth cookies.

3.7 `POST /api/public/auth/Logout` (`[AllowAnonymous]`)

Clears `RefreshTokenHash` & `PreviousRefreshTokenHash` in `dbo.Tbl_Reader` and deletes `reader_access_token` & `reader_refresh_token` cookies.

---

4. Production TypeScript / Axios Client (Mutex Queue + Selective Auth)

```
import axios, { AxiosError, InternalAxiosRequestConfig } from 'axios';

export const publicApi = axios.create({
  baseURL: import.meta.env.VITE_PUBLIC_API_URL ?? 'http://localhost:5080',
  withCredentials: true, // Sends reader_access_token & reader_refresh_token HttpOnly cookies
  headers: {
    'Content-Type': 'application/json',
  },
});

interface RetryableRequestConfig extends InternalAxiosRequestConfig {
  _retry?: boolean;
}

interface FailedQueueItem {
  resolve: () => void;
  reject: (error: unknown) => void;
}

let isRefreshing = false;
let failedQueue: FailedQueueItem[] = [];

const processQueue = (error: unknown = null) => {
  failedQueue.forEach(({ resolve, reject }) => {
    if (error) {
      reject(error);
    } else {
      resolve();
    }
  });
  failedQueue = [];
};

publicApi.interceptors.response.use(
  (response) => response,
  async (error: AxiosError) => {
    const originalRequest = error.config as RetryableRequestConfig | undefined;

    if (!originalRequest || error.response?.status !== 401) {
      return Promise.reject(error);
    }

    // Do not intercept 401s coming from Auth endpoints themselves
    const url = originalRequest.url ?? '';
    if (
      originalRequest._retry ||
      url.includes('/api/public/auth/Login') ||
      url.includes('/api/public/auth/Register') ||
      url.includes('/api/public/auth/Refresh') ||
      url.includes('/api/public/auth/Logout')
    ) {
      return Promise.reject(error);
    }

    if (isRefreshing) {
      return new Promise<void>((resolve, reject) => {
        failedQueue.push({ resolve, reject });
      }).then(() => publicApi(originalRequest));
    }
```

```
    originalRequest._retry = true;
    isRefreshing = true;

    try {
      const refreshRes = await publicApi.post('/api/public/auth/Refresh', {});
      if (!refreshRes.data?.IsSuccess) {
        throw new Error(refreshRes.data?.RespDesp || 'Session expired');
      }

      processQueue(null);
      return publicApi(originalRequest);
    } catch (refreshError) {
      processQueue(refreshError);
      window.dispatchEvent(new CustomEvent('reader:unauthorized'));
      return Promise.reject(refreshError);
    } finally {
      isRefreshing = false;
    }
  }
);
```